

CSA15 Cloud Computing and Big Data Analytics

CO3 AT2 - Capstone Cloud Access and Service Integration Case Study Student Template

Course Code	CSA15	Course Title	Cloud Computing and Big Data Analytics
Assessment	CO3 AT2 - Capstone Cloud Access and Service Integration Case Study	Submission Type	Individual digital case-study document
Faculty	Dr C. Rajesh Babu	Employee ID	SSETSCS415
Mode	Case study / capstone design reasoning	Submission	GitHub repository link through Google Classroom

How This Template Works

Read each case, connect it to the technical ideas learned in CO3 AT1, and then extend the design to your own capstone product. The answer must show cloud access, APIs, storage, serverless/event flow and edge/fog/cloud reasoning. Do not upload passwords, private keys, API tokens or sensitive personal data.

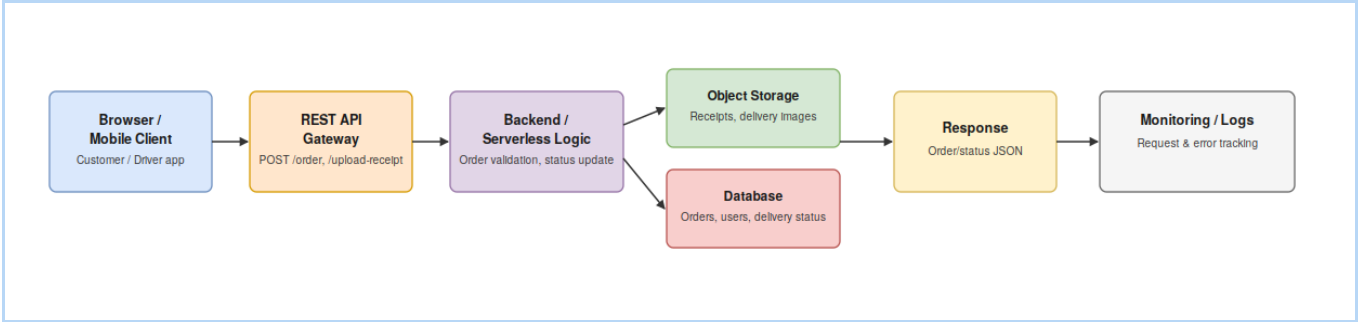
Student and Capstone Details

Student Name	D.S.Tharuniya Priya
Register Number	192511317
Class / Slot	Slot B
Capstone Title	PulsePay : Fake Screenshot detection using cloud computing
CO3 AT1 Evidence Link	https://github.com/tharu-ui/Cloud-Computing-and-BDA/tree/main/PulsePay/CO3-
GitHub Repository Link	https://github.com/tharu-ui/Cloud-Computing-and-BDA/tree/main/PulsePay/CO3-AT1-Cloud-Access-Experiments
Submission Date	06.08.2026

Case 1: Cloud-Native Backend and Web API Architecture

Scenario
A food delivery company wants to deploy a cloud-native backend with web APIs, browser/mobile clients and shared cloud storage for receipts and delivery images. Design the architecture and explain the role of each component.

A. Case Architecture Diagram



B. Component Role Explanation

The browser/mobile client is the delivery customer or driver's app, sending requests over HTTPS. The API layer exposes REST endpoints (e.g. POST /order, POST /upload-receipt) and enforces authentication before any request reaches business logic. The backend/serverless logic validates orders, processes delivery status updates, and handles receipt/image uploads. Shared cloud object storage holds receipts and delivery images as immutable files, referenced by database records rather than stored inside the database itself — this keeps the database lean and lets images scale independently. A relational or NoSQL database stores structured data (orders, users, delivery status) that needs querying, unlike the images which are only ever fetched by reference.

C. Capstone Extension: user-to-cloud workflow for your product

In PulsePay, the same pattern applies to fake UPI screenshot detection. The merchant/user uploads a payment screenshot from the browser or mobile client. The request hits a Cloud Function HTTP endpoint (API layer) which validates the Firebase Auth token before accepting the file. The backend logic (Cloud Function/Cloud Run) stores the raw image in Cloud Storage, then invokes Cloud Vision API to extract text and detect tampering signals. The verification result (genuine/fake, confidence score, transaction reference) is written to Firestore, not the image itself — Firestore holds only structured metadata and a reference link to the Cloud Storage object. The client then polls or receives the verification response through the API layer.

Case 2: Storage and Serverless Upload Workflow

Scenario
A media startup must decide between SAN-backed storage, object storage and serverless processing for upload workflows. Recommend a deployment model and justify it.

Data / Event	Best Storage	Serverless Action	Reason
Uploaded raw media file	Object Storage	Cloud Function triggered on upload event to start transcoding/processing	Object storage handles large unstructured binary files at low cost with

			native event triggers
Processed/transcoded output	Object Storage (separate output bucket, CDN-fronted)	Function writes processed file, updates status	Keeps raw and processed files separated for lifecycle/versioning control
File metadata (owner, timestamp, status, size)	NoSQL/Relational Database (Firestore)	Function updates DB record after processing completes	Metadata is structured and queryable; doesn't belong in object storage
Thumbnail/preview generation	Object Storage	Function generates thumbnail on upload trigger	Small derived files still fit the immutable-object pattern, generated on demand

A. Storage and Serverless Recommendation

Object storage combined with serverless processing is recommended over SAN-backed storage. SAN is block-level storage optimized for low-latency structured workloads like databases and VM disks — it is expensive to scale and not designed for large volumes of unstructured media. Object storage scales horizontally, is cost-efficient per GB, and integrates directly with event-driven serverless functions (upload triggers), which removes the need for an always-on server to watch for new files.

B. Failure Handling and Cost/Performance Explanation

Failure handling: each serverless function should be idempotent (safe to re-run if triggered twice), with retries and a dead-letter queue/topic for failed invocations so no upload is silently lost. Object storage versioning also protects against accidental overwrite.

Cost/Performance: object storage costs a fraction of SAN per GB and bills only for what is stored; serverless functions bill per invocation/duration instead of running a always-on compute instance, so idle time costs nothing. Trade-off is cold-start latency on serverless functions versus the near-instant response of SAN, which is acceptable here since uploads are not real-time transactional operations.

C. Capstone Extension: one upload/event workflow

User uploads a UPI payment screenshot → file lands in Cloud Storage bucket → this upload event triggers a Cloud Function → the function calls Cloud Vision API to run OCR and image-tamper checks → result and metadata are written to Firestore → a notification/response is returned to the client confirming genuine or fake status.

Case 3: Edge, Fog and Cloud Placement

Scenario

A smart-city platform needs low-latency decision-making near sensors while still using the cloud for analytics. Decide what runs at edge, fog and cloud.

Task	Layer	Latency Need	Data Volume	Reason
Traffic signal / motion sensor decision	Edge	Very low	Low	Immediate local action required, cannot wait for round trip to cloud
Aggregating readings from a cluster of sensors in one zone	Fog	Low - medium	Medium	Fog node consolidates nearby edge data before forwarding, cuts bandwidth to cloud
Real-time incident/accident alert generation	Edge/fog	Low	Low – medium	Needs fast local response but slightly more processing than a single sensor can do
Long-term analytics, historical trends, ML model training	cloud	High tolerance	large	Cloud has the storage and compute scale for big data processing that isn't time-critical

A. Edge/Fog/Cloud Architecture Explanation

The edge layer sits closest to the sensor and handles decisions needing millisecond response, such as an immediate traffic light change. The fog layer sits one hop back, aggregating and pre-processing data from multiple edge devices in a locality before sending a smaller, summarized payload to the cloud, reducing bandwidth and cloud compute cost. The cloud layer handles everything that can tolerate latency: historical analytics, dashboards, and training ML models on aggregated city-wide data.

B. Capstone Extension or Direct Cloud Access Justification

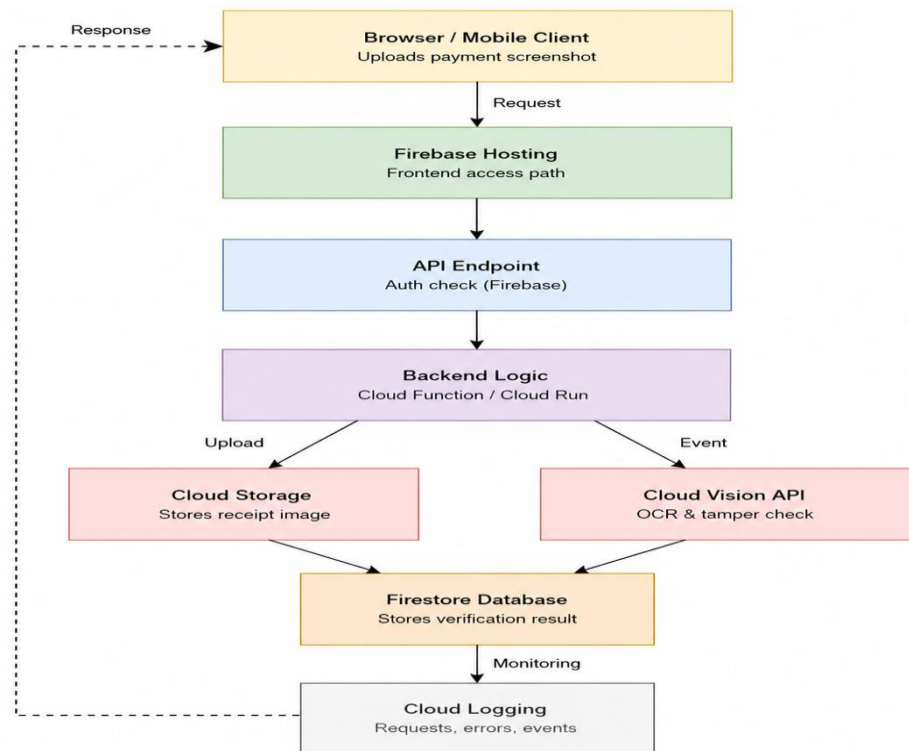
PulsePay does not need an edge or fog layer. Screenshot verification is an asynchronous, on-demand task — a user uploads an image and expects a result in seconds, not milliseconds, and there is no continuous sensor stream to pre-process locally. Direct cloud access (client → Cloud Function → Cloud Vision API → Firestore) is sufficient and simpler, since introducing edge/fog layers would add infrastructure cost with no latency benefit for this use case.

Final Capstone Cloud Access Architecture

- ☐ Show Browser/Mobile Client.
- ☐ Show hosted frontend or app access path.

- ☐ Show REST API endpoint and authentication check.
- ☐ Show backend/serverless logic.
- ☐ Show database, object/blob storage, logs and monitoring.
- ☐ Label the arrows: request, response, upload, event, notification or monitoring signal.

Final Diagram and Explanation



- **Browser/Mobile Client:** the merchant or user's app used to capture and upload the UPI payment screenshot; also the endpoint that receives the final genuine/fake verification result.
- **Hosted frontend/app access path:** the client app is served through Firebase Hosting, giving users a single URL to access PulsePay from browser or mobile without managing separate servers.
- **REST API endpoint and authentication check:** a Cloud Function exposed as an HTTP endpoint (e.g. POST /upload-receipt) receives the request. Before any business logic runs, it verifies the Firebase Auth token attached to the request, rejecting unauthenticated calls.
- **Backend/serverless logic:** once authenticated, the Cloud Function (or Cloud Run service for heavier processing) handles the core logic — it stores the incoming image, invokes Cloud Vision API for OCR and tamper analysis, and determines a genuine/fake result.
- **Database (Firestore):** stores structured, queryable data only — verification result, confidence score, timestamp, user ID, and a reference link to the image. It does not store the image itself.
- **Object/blob storage (Cloud Storage):** holds the actual uploaded screenshot as an immutable file. Keeping it separate from Firestore keeps the database fast and lightweight while storage scales independently for large files.
- **Logs and monitoring (Cloud Logging/Monitoring):** every function invocation, error, and authentication failure is logged automatically, giving visibility into system health and helping debug failed uploads or verification errors.

Submission Checklist

- ☐ Template completed in DOCX format.
- ☐ Diagrams pasted clearly in the document.
- ☐ GitHub repository contains DOCX/PDF, diagrams and README.
- ☐ Google Classroom submission contains GitHub link.
- ☐ Sensitive credentials are not visible anywhere.

Student Assurance

Declaration

I confirm that this case-study answer, cloud access design, diagrams and capstone extensions were prepared individually by me for the stated capstone title.

Student Signature: D.S.Tharuniya Priya **Date:** 07.08.2020